

Neostats : Use Case

Name : Jesvin Saji

Step 1: Data Understanding

What we are doing in this step is to understand what the dataset is. So before applying any transformations we will be understanding this Airline dataset.

```
import pandas as pd
import numpy as np

df = pd.read_excel("UseCase - Airlines.xlsx")

df.head()
```

	flight_id	airline	source	destination	departure_time	arrival_time	duration
0	SI010	SpiceJet	CCU	MAA	2026-04-20 23:38:41.701	2026-04-21 02:32:41.701	02:54:00
1	AI155	Air India	BOM	CCU	2026-04-20 23:35:41.703	2026-04-21 01:23:41.703	01:48:00
2	UK094	Vistara	BOM	CCU	2026-04-20 23:26:41.702	2026-04-21 01:11:41.702	01:45:00
3	AI245	Air India	BOM	CCU	2026-04-20 23:07:41.704	2026-04-21 01:43:41.704	02:36:00
4	AI192	Air India	MAA	BOM	2026-04-20 23:05:41.703	2026-04-21 04:04:41.703	04:59:00

We are reading the dataset and then displaying the first 5 values.

Dataset dimensions : 1020 ,7

```
df.columns

Index(['flight_id', 'airline', 'source', 'destination', 'departure_time',
       'arrival_time', 'duration'],
      dtype='object')
```

We are printing the column names.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1020 entries, 0 to 1019
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   flight_id       1020 non-null  object 
1   airline         979 non-null   object 
2   source          1020 non-null  object 
3   destination     1020 non-null  object 
4   departure_time  1020 non-null  datetime64[ns]
5   arrival_time   1020 non-null  datetime64[ns]
6   duration        1020 non-null  object 
dtypes: datetime64[ns](2), object(5)
memory usage: 55.9+ KB
```

Basic information like datatypes of the fields inside the dataset

```
df.isnull().sum()
[7] ✓ 0.0s
... flight_id      0
    airline      41
    source       0
    destination   0
    departure_time 0
    arrival_time  0
    duration      0
    dtype: int64
```

Missing values identified

Now we will get the count of Airlines , Source and Destinations:

```
df["airline"].value_counts(dropna=False)
[9] ✓ 0.0s
...
airline
IndiGo      249
SpiceJet    240
Air India   236
Vistara     223
NaN         41
UNKNOWN     31
Name: count, dtype: int64
```

```
df["source"].value_counts(dropna=False)
[10] ✓ 0.0s
...
source
BOM      207
HYD      180
CCU      174
DEL      163
MAA      157
BLR      139
Name: count, dtype: int64
```

```
df["destination"].value_counts(dropna=False)
[11] ✓ 0.0s
...
destination
DEL      200
CCU      188
BOM      174
BLR      165
MAA      151
HYD      142
Name: count, dtype: int64
```

These screenshots show us the count of Airlines, Source and Destinations of each value in the dataset

```
df["flight_id"].head(20)

0    S7010
1    AI155
2    UK094
3    AI245
4    AI192
5    S7158
6    6F196
7    AI080
8    6F025
9    6F251
10   AI137
11   6F026
12   UK167
13   UK027
14   AI069
15   AI058
16   AI140
17   S7239
18   6F099
19   AI048
Name: flight_id, dtype: object
```

Now in this screenshot we can see the first 20 flight id and its format
Unique flight ids found: 1004

```
print("Departure times:")
print(df["departure_time"].head(20))

print("\nArrival times:")
print(df["arrival_time"].head(20))

print("\nDuration values:")
print(df["duration"].head(20))

... Departure times:
0    2026-04-20 23:38:41.701
1    2026-04-20 23:35:41.703
2    2026-04-20 23:26:41.702
3    2026-04-20 23:07:41.704
4    2026-04-20 23:05:41.703
5    2026-04-20 23:05:41.703
6    2026-04-20 23:04:41.703
7    2026-04-20 23:03:41.702
8    2026-04-20 23:02:41.701
9    2026-04-20 22:56:41.703
10   2026-04-20 22:49:41.702
11   2026-04-20 22:46:42.000
12   2026-04-20 22:45:41.703
13   2026-04-20 22:44:41.701
14   2026-04-20 22:43:41.702
15   2026-04-20 22:40:41.701
16   2026-04-20 22:25:41.702
17   2026-04-20 22:24:41.704
18   2026-04-20 22:24:41.702
19   2026-04-20 22:23:41.701
Name: departure_time, dtype: datetime64[ns]
```

Here we can see the time formats of the first 20 values for each departure, arrival and duration... Since I am using the VS Code IDE it is not showing all the values in the screenshot

```
profile = pd.DataFrame({
    "column": df.columns,
    "data_type": df.dtypes.astype(str).values,
    "missing_values": df.isnull().sum().values,
    "unique_values": df.nunique(dropna=False).values
})

profile

...

```

	column	data_type	missing_values	unique_values
0	flight_id	object	0	1004
1	airline	object	41	6
2	source	object	0	6
3	destination	object	0	6
4	departure_time	datetime64[ns]	0	981
5	arrival_time	datetime64[ns]	0	969
6	duration	object	0	270

A basic profiling from the given dataset is shown in this screenshot

With this understanding we conclude Step 1.

Step 2: Identifying the problems before Cleaning

```
flight_id
6F250    2
AI031    2
UK163    2
AI242    2
SJ142    2
SJ146    2
AI020    2
SJ118    2
AI043    2
AI070    2
UK013    2
UK139    2
UK160    2
UK180    2
UK049    2
SJ037    2
Name: count, dtype: int64
```

Here we are looking at the duplicate flight id entries in the dataset. There are multiple entries with same flight id here

```
df[df["airline"].isna()]
```

[20] ✓ 0.0s

	flight_id	airline	source	destination	departure_time	arrival_time	duration
57	UK209	NaN	MAA	BLR	2026-04-20 19:02:41.704	2026-04-20 21:47:41.704	02:45:00
63	6F270	NaN	HYD	MAA	2026-04-20 18:14:41.701	2026-04-20 19:02:41.701	00:48:00
89	UK224	NaN	CCU	MAA	2026-04-20 16:25:41.704	2026-04-20 20:24:41.704	03:59:00
118	6F261	NaN	CCU	DEL	2026-04-20 14:31:41.703	2026-04-20 17:29:41.703	02:58:00
120	6F254	NaN	MAA	BLR	2026-04-20 14:26:41.703	2026-04-20 18:01:41.703	03:35:00
124	6F259	NaN	BOM	DEL	2026-04-20 14:10:41.703	2026-04-20 15:21:41.703	01:11:00
133	UK141	NaN	HYD	MAA	2026-04-20 13:38:41.703	2026-04-20 15:59:41.703	02:21:00
150	6F255	NaN	DEL	BOM	2026-04-20 11:59:41.703	2026-04-20 15:33:41.703	03:34:00
218	AI047	NaN	CCU	MAA	2026-04-20 06:36:41.701	2026-04-20 07:23:41.701	00:47:00
222	AI128	NaN	BOM	CCU	2026-04-20 06:07:41.702	2026-04-20 10:03:41.702	03:56:00
225	SJ138	NaN	BLR	BOM	2026-04-20 05:55:41.702	2026-04-20 06:43:41.702	00:48:00
279	AI009	NaN	DEL	CCU	2026-04-20 01:31:41.701	2026-04-20 04:05:41.701	02:34:00
284	AI226	NaN	BLR	HYD	2026-04-20 01:19:41.703	2026-04-20 04:08:41.703	02:49:00
299	6F272	NaN	HYD	MAA	2026-04-19 23:35:41.702	2026-04-20 00:22:41.702	00:47:00
314	SJ099	NaN	MAA	BLR	2026-04-19 22:14:41.702	2026-04-20 00:34:41.702	02:20:00
319	6F260	NaN	CCU	DEL	2026-04-19 21:51:41.702	2026-04-20 01:14:41.702	03:23:00
351	6F271	NaN	HYD	MAA	2026-04-19 19:33:41.702	2026-04-19 22:52:41.702	03:19:00

```
df[df["airline"] == "UNKNOWN"]
```

[21] ✓ 0.0s

	flight_id	airline	source	destination	departure_time	arrival_time	duration
9	6F251	UNKNOWN	DEL	BLR	2026-04-20 22:56:41.703	2026-04-20 23:37:41.703	00:41:00
14	AI069	UNKNOWN	DEL	BOM	2026-04-20 22:43:41.702	2026-04-21 02:29:41.702	03:46:00
28	UK003	UNKNOWN	HYD	DEL	2026-04-20 21:41:41.700	2026-04-21 02:19:41.700	04:38:00
58	SJ043	UNKNOWN	MAA	DEL	2026-04-20 18:59:41.701	2026-04-20 21:08:41.701	02:09:00
83	AI242	UNKNOWN	BLR	CCU	2026-04-20 16:41:41.704	2026-04-20 17:43:41.704	01:02:00
84	AI242	UNKNOWN	BLR	CCU	2026-04-20 16:41:41.704	2026-04-20 17:43:41.704	01:02:00
90	UK077	UNKNOWN	HYD	BOM	2026-04-20 16:22:41.702	2026-04-20 18:00:41.702	01:38:00
148	AI222	UNKNOWN	BOM	DEL	2026-04-20 12:10:41.703	2026-04-20 13:14:41.703	01:04:00
189	AI044	UNKNOWN	MAA	BLR	2026-04-20 09:03:41.701	2026-04-20 12:36:41.701	03:33:00
209	UK229	UNKNOWN	HYD	DEL	2026-04-20 07:40:41.704	2026-04-20 11:57:41.704	04:17:00
220	AI088	UNKNOWN	MAA	DEL	2026-04-20 06:34:41.702	2026-04-20 10:10:41.702	03:36:00
253	6F250	UNKNOWN	DEL	BLR	2026-04-20 03:26:41.701	2026-04-20 07:30:41.701	04:04:00
270	6F250	UNKNOWN	CCU	BLR	2026-04-20 02:23:41.702	2026-04-20 02:56:41.702	00:33:00

Here we are looking at two different cases..

1st screenshot says the missing airlines and the 2nd screenshot says the details of airlines which are UNKNOWN...All the values are not printed in the screenshot.

```
same_location = df[df["source"] == df["destination"]]

same_location

[24] ✓ 0.0s

... flight_id airline source destination departure_time arrival_time duration
```

We are not getting any values for source= destinations.

```
df["calculated_duration"] = df["arrival_time"] - df["departure_time"]

df[["flight_id",
    "departure_time",
    "arrival_time",
    "duration",
    "calculated_duration"
]].head(20)

✓ 0.0s Python
```

	flight_id	departure_time	arrival_time	duration	calculated_duration
0	SJ010	2026-04-20 23:38:41.701	2026-04-21 02:32:41.701	02:54:00	0 days 02:54:00
1	AI155	2026-04-20 23:35:41.703	2026-04-21 01:23:41.703	01:48:00	0 days 01:48:00
2	UK094	2026-04-20 23:26:41.702	2026-04-21 01:11:41.702	01:45:00	0 days 01:45:00
3	AI245	2026-04-20 23:07:41.704	2026-04-21 01:43:41.704	02:36:00	0 days 02:36:00
4	AI192	2026-04-20 23:05:41.703	2026-04-21 04:04:41.703	04:59:00	0 days 04:59:00
5	SJ158	2026-04-20 23:05:41.703	2026-04-21 01:24:41.703	02:19:00	0 days 02:19:00
6	6F196	2026-04-20 23:04:41.703	2026-04-21 00:47:41.703	01:43:00	0 days 01:43:00
7	AI080	2026-04-20 23:03:41.702	2026-04-21 00:35:41.702	01:32:00	0 days 01:32:00
8	6F025	2026-04-20 23:02:41.701	2026-04-20 23:57:41.701	00:55:00	0 days 00:55:00
9	6F251	2026-04-20 22:56:41.703	2026-04-20 23:37:41.703	00:41:00	0 days 00:41:00
10	AI137	2026-04-20 22:49:41.702	2026-04-21 01:17:41.702	02:28:00	0 days 02:28:00
11	6F026	2026-04-20 22:46:42.000	2026-04-21 03:13:42.000	04:27:00	0 days 04:27:00
12	UK167	2026-04-20 22:45:41.703	2026-04-21 01:46:41.703	03:01:00	0 days 03:01:00
13	UK027	2026-04-20 22:44:41.701	2026-04-21 01:05:41.701	02:21:00	0 days 02:21:00
14	AI069	2026-04-20 22:43:41.702	2026-04-21 02:29:41.702	03:46:00	0 days 03:46:00
15	AI058	2026-04-20 22:40:41.701	2026-04-21 00:18:41.701	01:38:00	0 days 01:38:00

just check suration by creating a field to view the duration in a better way .Not all values are printed in the screenshot.

```
overnight_flights = df[
    df["arrival_time"].dt.date > df["departure_time"].dt.date
]

overnight_flights[["flight_id",
    "departure_time",
    "arrival_time",
    "duration"
]]

[26] ✓ 0.0s Python
```

	flight_id	departure_time	arrival_time	duration
0	SJ010	2026-04-20 23:38:41.701	2026-04-21 02:32:41.701	02:54:00
1	AI155	2026-04-20 23:35:41.703	2026-04-21 01:23:41.703	01:48:00
2	UK094	2026-04-20 23:26:41.702	2026-04-21 01:11:41.702	01:45:00
3	AI245	2026-04-20 23:07:41.704	2026-04-21 01:43:41.704	02:36:00
4	AI192	2026-04-20 23:05:41.703	2026-04-21 04:04:41.703	04:59:00
...	...	...	...	...
916	6F242	2026-04-17 21:10:41.704	2026-04-18 01:04:41.704	03:54:00
918	SJ124	2026-04-17 21:05:41.702	2026-04-18 01:12:41.702	04:07:00
924	AI230	2026-04-17 20:37:41.704	2026-04-18 00:22:41.704	03:45:00
926	AI124	2026-04-17 20:28:41.702	2026-04-18 00:00:41.702	03:32:00
932	6F147	2026-04-17 19:57:41.703	2026-04-18 00:31:41.703	04:34:00

124 rows x 4 columns

Checked for overnight flights and found 124 of them.

```
df[df["duration"].apply(lambda x: isinstance(x, datetime.datetime))]
```

```
[36] ✓ 0.0s
```

	flight_id	airline	source	destination	departure_time	arrival_time	duration	calculated_duration
355	SJ192	SpiceJet	HYD	BOM	2026-04-19 18:45:42	2026-04-18 23:45:42	1899-12-29 05:00:00	-1 days +05:00:00

Found a anomaly for flight with flight id: SJ192: The arrival time < departure time.

```

...
Missing values:
flight_id      0
airline        41
source         0
destination    0
departure_time 0
arrival_time   0
duration       0
calculated_duration
duration_td    0
dtype: int64

Exact duplicate rows: 15

Duplicate flight IDs: 16

Airline values:
airline
Indigo      249
SpiceJet    240
Air India   236
Vistara     223
NaN         41
UNKNOWN    31
Name: count, dtype: int64

```

A final cell that shows the anomalies that were founded in the dataset.

Step 3: Data Cleaning

```

df_clean = df_clean.drop_duplicates()

print("Shape after removing exact duplicates:", df_clean.shape)
[49] ✓ 0.0s
... Shape after removing exact duplicates: (1005, 9)

```

First we are removing the duplicate values

We can see that the shape of the dataset after duplicate value removal is 1005,9

```

text_columns = ["flight_id", "airline", "source", "destination"]

for col in text_columns:
    df_clean[col] = df_clean[col].astype("string").str.strip()

print(df_clean[text_columns].head())
[50] ✓ 0.0s
...
flight_id  airline  source  destination
0      S3010  SpiceJet  CCU      MAA
1      AI155  Air India  BOM      CCU
2      UK094  Vistara   BOM      CCU
3      AI245  Air India  BOM      CCU
4      AI192  Air India  MAA      BOM

```

Removing white spaces from names. We wont be removing white space from valid Airline names like Air India. This step is basic Standardization

```

df_clean["airline"] = df_clean["airline"].fillna("UNKNOWN")

print(df_clean["airline"].value_counts(dropna=False))
print("\nMissing airline values:", df_clean["airline"].isna().sum())
[51] ✓ 0.0s
...
airline
Indigo      249
SpiceJet    236
Air India   233
Vistara     218
UNKNOWN     69
Name: count, dtype: Int64

Missing airline values: 0

```

We are handling the missing values. Now the missing values are dealt.

```

df_clean["duration_clean"] = df_clean["duration"].apply(
    lambda x: pd.Timedelta(
        hours=x.hour,
        minutes=x.minute,
        seconds=x.second
    ) if isinstance(x, datetime.time)
    else pd.Timedelta(
        days=x.day - 1,
        hours=x.hour,
        minutes=x.minute,
        seconds=x.second
    )
)

print(df_clean["duration_clean"].head())
print("\nduration data type:", df_clean["duration_clean"].dtype)

```

[42] ✓ 0.0s Python

```

... 0 0 days 02:54:00
    1 0 days 01:48:00
    2 0 days 01:45:00
    3 0 days 02:36:00
    4 0 days 04:59:00
Name: duration_clean, dtype: timedelta64[ns]
Duration data type: timedelta64[ns]

```

Since the datatype of duration was mixed which consisted of both date and datetime datatypes, we are assigning a proper Timedelta for each value regardless they are any datatype.

```

invalid_time = df_clean["arrival_time"] < df_clean["departure_time"]

df_clean.loc[invalid_time, "arrival_time"] = (
    df_clean.loc[invalid_time, "arrival_time"] + pd.Timedelta(days=1)
)

print("Invalid timestamp orders remaining:",
      (df_clean["arrival_time"] < df_clean["departure_time"]).sum())

```

[53] ✓ 0.0s

```

... Invalid timestamp orders remaining: 0

```

There was exactly one entry in the dataset with wrong duration due to arrival being earlier than departure. In here we are dealing with that problem by adding 1 day to the arrival time. The consistent duration that was supposed to be there was 5 hours but instead it was -1 day +5:00. So we are adding 1 day to the arrival time and get a consistent duration of 5 hrs.

```

df_clean["calculated_duration"] = (
    df_clean["arrival_time"] - df_clean["departure_time"]
)

print(df_clean[
    "flight_id",
    "departure_time",
    "arrival_time",
    "duration_clean",
    "calculated_duration"
].head())

```

[54] ✓ 0.0s

```

... flight_id  departure_time  arrival_time  duration_clean \
0  SJ1010  2026-04-20 23:38:41.701  2026-04-21 02:32:41.701  0 days 02:54:00
1  AI1155  2026-04-20 23:35:41.703  2026-04-21 01:23:41.703  0 days 01:48:00
2  UK094  2026-04-20 23:26:41.702  2026-04-21 01:11:41.702  0 days 01:45:00
3  AI245  2026-04-20 23:07:41.704  2026-04-21 01:43:41.704  0 days 02:36:00
4  AI1192  2026-04-20 23:05:41.703  2026-04-21 04:04:41.703  0 days 04:59:00

calculated_duration
0  0 days 02:54:00
1  0 days 01:48:00
2  0 days 01:45:00
3  0 days 02:36:00
4  0 days 04:59:00

```

Recalculating duration for including the fixed flight id too.

Step 4: Validation

```
df_clean["flight_duration"] = (
    df_clean["calculated_duration"].dt.total_seconds() / 60
).round().astype(int)

print(df_clean[["flight_id",
                "duration_clean",
                "calculated_duration",
                "flight_duration"]].head())
```

```
[58] ✓ 0.0s
```

	flight_id	duration_clean	calculated_duration	flight_duration
0	57010	0 days 02:54:00	0 days 02:54:00	174
1	AI155	0 days 01:48:00	0 days 01:48:00	108
2	UK094	0 days 01:45:00	0 days 01:45:00	105
3	AI245	0 days 02:36:00	0 days 02:36:00	156
4	AI192	0 days 04:59:00	0 days 04:59:00	299

Checking weather any flight has 0 or negative durations

```
print("Minimum duration:", df_clean["flight_duration"].min(), "minutes")
print("Maximum duration:", df_clean["flight_duration"].max(), "minutes")

print("Invalid durations:",
      (df_clean["flight_duration"] <= 0).sum())
```

```
[59] ✓ 0.0s Python
```

```
... Minimum duration: 30 minutes
Maximum duration: 300 minutes
Invalid durations: 0
```

```
print("Unique sources:", df_clean["source"].unique())
print("Unique destinations:", df_clean["destination"].unique())

same_route = (df_clean["source"] == df_clean["destination"]).sum()
print("Same source/destination flights:", same_route)
```

```
[60] ✓ 0.0s Python
```

```
... Unique sources: <StringArray>
['CCU', 'BOM', 'MAA', 'DEL', 'BLR', 'HYD']
Length: 6, dtype: string
Unique destinations: <StringArray>
['MAA', 'CCU', 'BOM', 'HYD', 'BLR', 'DEL']
Length: 6, dtype: string
Same source/destination flights: 0
```

Two steps are done in this screenshot

1st :Checking maximum and minimum duration to avoid invalid durations

2nd :Checking whether there is same arrival and departure location .

```
df_clean["route"] = (
    df_clean["source"] + " → " + df_clean["destination"]
)

print(df_clean[["source", "destination", "route"]].head())
```

```
[61] ✓ 0.0s Python
```

	source	destination	route
0	CCU	MAA	CCU → MAA
1	BOM	CCU	BOM → CCU
2	BOM	CCU	BOM → CCU
3	BOM	CCU	BOM → CCU
4	MAA	BOM	MAA → BOM

Creating a user friendly field called route.

```
df_clean["airline_standardized"] = (
    df_clean["airline"]
    .str.upper()
    .str.strip()
)

print(df_clean["airline_standardized"].value_counts())
```

```
[62] ✓ 0.0s
```

```
... airline_standardized
INDIGO      249
SPIRITJET   236
AIR INDIA   233
VISTARA     218
UNKNOWN     69
Name: count, dtype: Int64
```

Validating all airlines and number of Unknown values.

```
df_clean["flight_duration_hours"] = (
    df_clean["flight_duration"] / 60
).round(2)

print(
    df_clean[
        ["flight_id", "flight_duration", "flight_duration_hours"]
    ].head()
)
```

[66] ✓ 0.0s

	flight_id	flight_duration	flight_duration_hours
0	SJ010	174	2.90
1	AI155	108	1.80
2	UK094	105	1.75
3	AI245	156	2.60
4	AI192	299	4.98

Converting duration into hours

```
Rows: 1005
Columns: 16

Missing values:
0

Exact duplicates: 0

Invalid timestamps: 0
Invalid durations: 0
Overnight flights: 122

Final columns:
['flight_id', 'airline', 'source', 'destination', 'departure_time', 'arrival_time', 'duration', 'calculated_duration', 'duration_td', 'duration_clean', 'flight_duration', 'route']
```

Overall validations done in this step

Step 5: KPI and Aggregation

```
avg_duration = df_clean["flight_duration_hours"].mean()

print(f"Average Flight Duration: {avg_duration:.2f} hours")
```

[69] ✓ 0.0s

.. Average Flight Duration: 2.74 hours

Creating an Average flight KPI

	route	flight count
6	BOM → CCU	90
12	CCU → DEL	72
25	MAA → BLR	65
0	BLR → BOM	60
24	HYD → MAA	57
18	DEL → HYD	54
23	HYD → DEL	42
7	BOM → DEL	39
11	CCU → BOM	33
15	DEL → BLR	29
16	DEL → BOM	28
21	HYD → BOM	27
9	BOM → MAA	27
8	BOM → HYD	26
28	MAA → DEL	26
22	HYD → CCU	26
17	DEL → CCU	26
20	HYD → BLR	26
27	MAA → CCU	24
14	CCU → MAA	24
5	BOM → BLR	23
19	DEL → MAA	23
1	BLR → CCU	21
13	CCU → HYD	21

Recording total number of flights for each routes .Not every route is shown in this screenshot.

```
airline_distribution = (
    df_clean.groupby("airline_standardized")
    .size()
    .reset_index(name="flight_count")
    .sort_values("flight_count", ascending=False)
)

airline_distribution
```

✓ 0.0s

	airline_standardized	flight_count
1	INDIGO	249
2	SPICEJET	236
0	AIR INDIA	233
4	VISTARA	218
3	UNKNOWN	69

Airline wise count KPI s

```
airline_duration = (
    df_clean.groupby("airline_standardized")["flight_duration_hours"]
    .mean()
    .reset_index(name="average_duration_hours")
    .sort_values("average_duration_hours", ascending=False)
)

airline_duration
```

✓ 0.0s

	airline_standardized	average_duration_hours
1	INDIGO	2.795261
3	UNKNOWN	2.767826
2	SPICEJET	2.736525
0	AIR INDIA	2.718841
4	VISTARA	2.711376

Average duration for each Airline .

```
anomaly_summary = {
    "Missing/Unknown Airlines": (df_clean["airline_standardized"] == "UNKNOWN").sum(),
    "Overnight flights": df_clean["is_overnight"].sum(),
    "Duplicate Flight IDs": df_clean["flight_id"].duplicated().sum()
}

anomaly_summary
```

✓ 0.0s

```
{'Missing/Unknown Airlines': np.int64(69),
 'Overnight flights': np.int64(122),
 'Duplicate Flight IDs': np.int64(1)}
```

Anomaly Summary shown in the above screenshot : There exist one duplicate flight id which has different routes. So we keep that in our cleaned dataset

```
df_clean.to_csv("cleaned_airlines.csv", index=False)

print("Cleaned dataset saved successfully.")
print("Final rows:", len(df_clean))
```

✓ 0.2s

Cleaned dataset saved successfully.
Final rows: 1005

Saving cleaned Dataset.

Step 6 : Data Visualizations

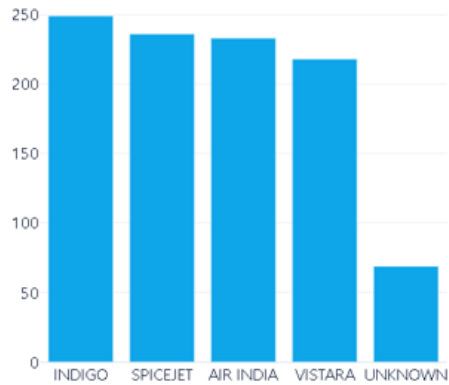
Due to hardware constraints I did not do the visualization in PowerBI as mentioned in the instructions. Instead I did the visualization using python library Dash and visualizations through plotly.

Screenshots:

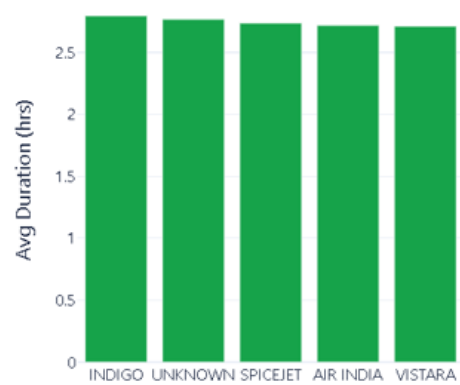


Airline Performance

Flights by Airline

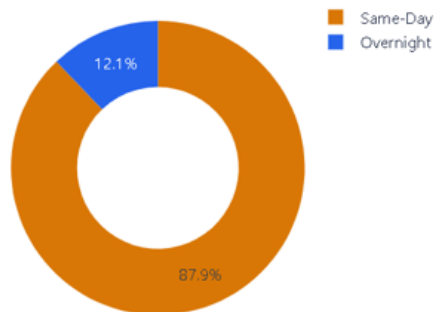


Avg Flight Duration by Airline



Overnight / Cross-Day Analysis

Overnight vs Same-Day Flights



Flights by Departure Date



Data Quality / Anomaly Insights

Issues identified during the data-engineering pipeline (not operational delay metrics)

Missing / Unknown Airlines

69

Duplicate Flight IDs

1

Overnight Flights

122

These are the KPI cards and graphs which are generated from the cleaned data

CONCLUSION

In this project I created a pipeline for the given Airline Dataset . In this pipeline the dataset travelled through various stages and was prepared for visualization.

The various stages were:-

Data Understanding – Problem Identification – Data Cleaning – Data Validation – KPI and Aggregation – Data Visualization

We properly found out which all problems were to be fixed in the 2nd phase and then they were properly handled in the 3rd phase.

I made sure that all the fields in the cleaned dataset are valid for the data visualization and KPI card creation in the 4th phase and then in the 5th phase I created fields required for KPI cards.

At the end I made app.py web app file using Flask, and the dashboard was created with the help of python libraries Plotly , Dash.

These are the key things that I have done in this project .